

编译原理

第五讲：语法分析之递归下降分析法

授课教师：柯嵩宇

福州大学计算机与大数据学院

2026 年 4 月 2 日



福州大学
FUZHOU UNIVERSITY 1958



- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业

目录



- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业

语法分析程序的功能

- 核心任务：检查词法分析器输出的符号串是否符合文法规则。
- 主要功能：
 - ① 结构识别：根据文法规则，将单词序列组合成各类语法短语（如表达式、语句等）。
 - ② 语法检查：发现并报告语法错误。
 - ③ 树构造：在某些实现中，直接构造抽象语法树（AST）。

示例： $a = b + c * 5$

语法分析器需要确认这是一个“赋值语句”，并识别出其中的算术优先级。

语法分析器的类型

自顶向下 (Top-Down)

自底向上 (Bottom-Up)



语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。

自底向上 (Bottom-Up)

语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。

自底向上 (Bottom-Up)

语法分析器的类型

自顶向下 (Top-Down)

- 从开始符号出发，寻找“匹配”的推导。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从根开始，构造语法树。

自底向上 (Bottom-Up)

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发，寻找“匹配”的**推导**。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从**根**开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发，寻找“匹配”的**推导**。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从**根**开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从**输入串**开始，逐步进行**归约**，直到文法的**开始符号**。

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发，寻找“匹配”的**推导**。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从**根**开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从**输入串**开始，逐步进行**归约**，直到文法的**开始符号**。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发，寻找“匹配”的**推导**。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从**根**开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从**输入串**开始，逐步进行**归约**，直到文法的**开始符号**。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从**叶节点**开始，构造语法树。

语法分析器的类型

自顶向下 (Top-Down)

- 从**开始符号**出发，寻找“匹配”的**推导**。
- 推导：根据文法的产生式规则，把串中出现的产生式的左部符号替换成右部。
- 从**根**开始，构造语法树。
- 具体实现：递归下降分析法、预测分析法。

自底向上 (Bottom-Up)

- 从**输入串**开始，逐步进行**归约**，直到文法的**开始符号**。
- 归约：根据文法的产生式规则，把串中出现的产生式的右部替换成左部符号。
- 从**叶节点**开始，构造语法树。
- 具体实现：算符优先分析法、LR 分析法。

目录

- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
 - 左递归
 - 回溯
 - 综合处理顺序
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业



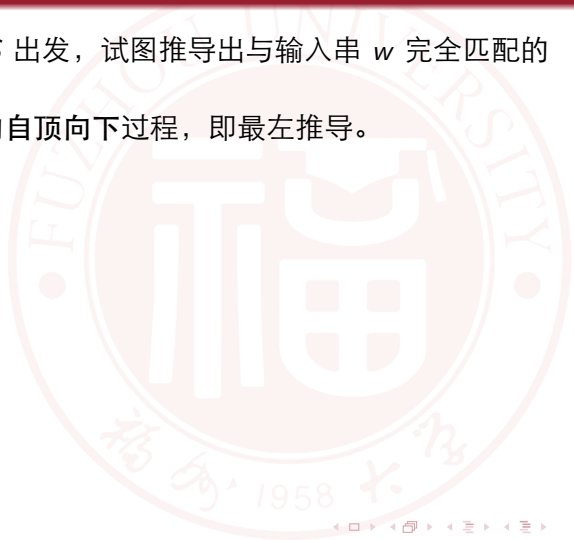
自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。



自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。



自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。

面临的挑战

- 左递归问题：可能导致推导进入死循环。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。

面临的挑战

- 左递归问题：可能导致推导进入死循环。
- 回溯开销：选择错误的产生式会导致效率低下。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。
- 典型方法：

面临的挑战

- 左递归问题：可能导致推导进入死循环。
- 回溯开销：选择错误的产生式会导致效率低下。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。
- 典型方法：
递归下降分析 对应文法每一个非终结符编写一个子程序。

面临的挑战

- 左递归问题：可能导致推导进入死循环。
- 回溯开销：选择错误的产生式会导致效率低下。

自顶向下分析法 (Top-Down Parsing)

- 基本思想：从文法的开始符号 S 出发，试图推导出与输入串 w 完全匹配的句子。
- 推导本质：对应于构造语法树的自顶向下过程，即最左推导。

关键操作

- 匹配 (**Match**)：终结符与输入符号一致，移动读头。
- 替换 (**Replace**)：非终结符根据产生式进行展开。

面临的挑战

- 左递归问题：可能导致推导进入死循环。
- 回溯开销：选择错误的产生式会导致效率低下。

- 典型方法：

递归下降分析 对应文法每一个非终结符编写一个子程序。

LL(1) 预测分析 利用预测分析表进行非递归的、无回溯的分析。

带回溯的自顶向下分析方法

- 策略：尝试所有可能的产生式。
- 回溯：若当前路径与输入不符，则退回读头，尝试下一个候选式。

示例： $S \rightarrow abc \mid abd$, 输入 abd

1. 匹配 $S \rightarrow abc$ ，前两个符 ab 成功。
2. 第三个符 $c \neq d$ ，匹配失败。
3. 回溯：读头退回 a 之前。
4. 匹配 $S \rightarrow abd$ ，成功。

目录

- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
 - 左递归
 - 回溯
 - 综合处理顺序
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业



Top-Down 分析器挑战一：左递归

- 在自顶向下分析器寻找匹配的推导时，通常采用的是最左推导的形式，即每次选择**最左边**的非终结符进行替换。
- 按顺序尝试非终结符的不同产生式，有可能出现最左侧符号不断出现的**无限循环**的情况。

文法

- $expr \rightarrow expr + term \mid expr - term \mid term$
- $term \rightarrow term * factor \mid term / factor \mid factor$
- $factor \rightarrow digit \mid (expr)$
- $digit \rightarrow 0 \mid 1 \mid \dots \mid 9$

目标句子：8+8+8

$expr$
 $\Rightarrow expr + term$
 $\Rightarrow expr + term + term$
 $\Rightarrow expr + term + term + term$
 $\Rightarrow \dots$

Top-Down 分析器挑战一：左递归

直接左递归

$$\begin{aligned} \text{expr} &\rightarrow \text{expr} + \text{term} | \text{expr} - \text{term} | \text{term} \\ \text{term} &\rightarrow \text{term} * \text{factor} | \text{term} / \text{factor} | \text{factor} \\ \text{factor} &\rightarrow \text{digit} | (\text{expr}) \\ \text{digit} &\rightarrow 0 | 1 | \dots | 9 \end{aligned}$$

间接左递归

$$\begin{aligned} S &\rightarrow AB \\ A &\rightarrow TC \\ T &\rightarrow S\gamma \end{aligned}$$

直接左递归的消除

有左递归文法

- $\text{expr} ::= \text{expr} + \text{term} \mid \text{expr} - \text{term} \mid \text{term}$
- $\text{term} ::= \text{term} * \text{factor} \mid \text{term} / \text{factor} \mid \text{factor}$
- $\text{factor} ::= \text{digit} \mid (\text{expr})$

无左递归文法

- $\text{expr} ::= \text{term expr}'$
- $\text{expr}' ::= + \text{term expr}' \mid - \text{term expr}' \mid \varepsilon$
- $\text{term} ::= \text{factor term}'$
- $\text{term}' ::= * \text{factor term}' \mid / \text{factor term}' \mid \varepsilon$
- $\text{factor} ::= \text{digit} \mid (\text{expr})$

消除直接左递归的算法

Algorithm 1: 消除直接左递归

Input: 文法 G , 非终结符 $\{A_1, A_2, \dots, A_n\}$

Result: 无直接左递归的文法 G'

```
1 foreach 非终结符  $A_i \in G$  do
2   将  $A_i$  的生成是划分为两类:
3   1. 有左递归的:  $A_i \rightarrow A_i\alpha_1 \mid A_i\alpha_2 \mid \dots \mid A_i\alpha_m$ 
4   2. 无左递归的:  $A_i \rightarrow \beta_1 \mid \beta_2 \mid \dots \mid \beta_k$ 
5   if  $m > 0$  then
6     创建一个新的非终结符  $A'_i$ ;
7     将  $A_i$  的生成是替换为:  $A_i \rightarrow \beta_1 A'_i \mid \beta_2 A'_i \mid \dots \mid \beta_k A'_i$ ;
8     为  $A'_i$  创建生成式:  $A'_i \rightarrow \alpha_1 A'_i \mid \alpha_2 A'_i \mid \dots \mid \alpha_m A'_i \mid \epsilon$ ;
9   end
10 end
```

间接左递归的消除

Algorithm 2: 消除间接左递归

Input: 按照某个顺序将非终结符号排序为 $A_1, A_2, \dots, A_n$

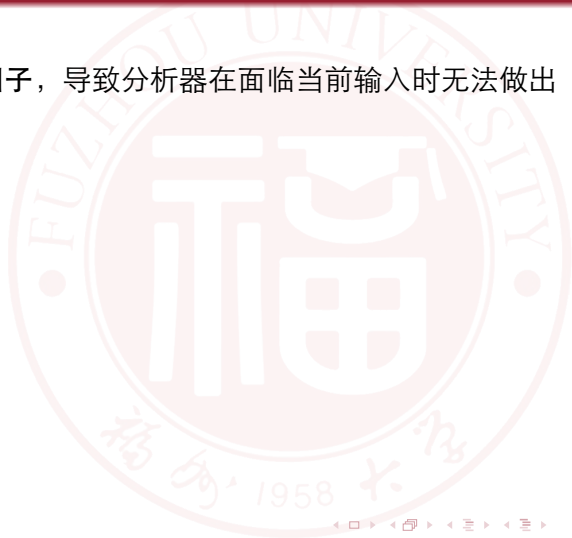
Result: 无简介左递归的文法

```

1 for  $i \leftarrow 1$  to  $n$  do
2   for  $j \leftarrow 1$  to  $i - 1$  do
3     消除由  $A_j$  导致的左递归, 即向每个形如  $A_i \rightarrow A_j \gamma$  的产生式代入所有  $A_j$ 
      为头的规则, 得到  $A_i \rightarrow \delta_1 \gamma | \delta_2 \gamma | \dots | \delta_k \gamma$ , 其中  $A_j \rightarrow \delta_1 | \delta_2 | \dots | \delta_k$ , 是所有的
       $A_j$  产生式;
      // 最坏情况会产生  $\mathcal{O}(n \times m)$  个新的产生式
4   end
5   消除  $A_i$  的直接左递归;
      // 方法类似, 同样的有可能产生  $\mathcal{O}(n \times m)$  个产生式
6 end
    
```

自顶向下分析挑战之二：回溯 (Backtracking)

- 问题的本质：文法存在公共左因子，导致分析器在面临当前输入时无法做出“唯一选择”。



自顶向下分析挑战之二：回溯 (Backtracking)

- 问题的本质：文法存在公共左因子，导致分析器在面临当前输入时无法做出“唯一选择”。

回溯的代价

若有 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ ，当输入匹配 α 后：

- 盲目性：只能按顺序尝试候选式。
- 副作用：匹配失败需回退输入指针（读头）、恢复递归栈状态。
- 复杂度：极端下导致时间复杂度爆炸。

自顶向下分析挑战之二：回溯 (Backtracking)

- 问题的本质：文法存在公共左因子，导致分析器在面临当前输入时无法做出“唯一选择”。

回溯的代价

若有 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ ，当输入匹配 α 后：

- 盲目性：只能按顺序尝试候选式。
- 副作用：匹配失败需回退输入指针（读头）、恢复递归栈状态。
- 复杂度：极端下导致时间复杂度爆炸。

提取左公因子 (Factoring)

将公共前缀 α 提取出来：

$$\begin{aligned} A &\rightarrow \alpha A' \\ A' &\rightarrow \beta_1 \mid \beta_2 \end{aligned}$$

效果：推迟决策。将选择点移至匹配完 α 之后。

目录

- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
 - 左递归
 - 回溯
 - 综合处理顺序
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业



Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow \underline{if\ (expr)\ then\ stmt\ else\ stmt} \\ &\quad | \quad \underline{if\ (expr)\ then\ stmt} \end{aligned}$$

Top-Down 分析器挑战二：回溯

如下文法该选择哪条规则？

$$\begin{aligned} stmt &\rightarrow \underline{if (expr) then stmt} \text{ else } stmt \\ &\quad | \quad \underline{if (expr) then stmt} \end{aligned}$$

手段：通过改写产生式来推迟决定，等读入了足够多的输入，获得足够信息后再做出正确的选择。

$$\begin{aligned} stmt &\rightarrow if (expr) then stmt \text{ else } stmt \\ elsestmt &\rightarrow \varepsilon \quad | \quad else \text{ } stmt \end{aligned}$$

Top-Down 分析器挑战二：回溯

算法：提取左公因式

Algorithm 3: 提取左公因式

Input: 按照某个顺序将非终结符号排序为 $A_1, A_2, \dots, A_n$

Result: 无左公因式的文法

- 1 对于每个非终结符 A ，找出它的两个或多个候选式的最长公共前缀 α 。若 $\alpha \neq \varepsilon$ ，则将所有 A -产生式 $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \dots | \alpha\beta_n | \gamma_1 | \gamma_2 | \dots | \gamma_m$ 改写为

$$\begin{aligned} A &\rightarrow \alpha A' | \gamma_1 | \gamma_2 | \dots | \gamma_m \\ A' &\rightarrow \beta_1 | \beta_2 | \dots | \beta_n \end{aligned}$$

其中， γ_i 表示所有不以 α 开头的产生式体；

- 2 不断应用这个转换，直到每个非终结符的任意两个产生式体都没有公共前缀为止；

提取左公因式

例

改写下列文法以便无回溯的自顶向下分析：

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow ab|a. \end{aligned}$$

解

对 A 提取左公因子得

$$\begin{aligned} S &\rightarrow cAd \\ A &\rightarrow aA' \\ A' &\rightarrow b|\varepsilon \end{aligned}$$

提取左公因式

例

改写文法 $S \rightarrow abcd|abce|abf$ 以便无回溯的自顶向下分析。

解

S 所有子式的最长左公因子为 ab ，引入 A 改写生成式得 $S \rightarrow abA, A \rightarrow cd|ce|f$ 。
再提取 A 的左公因子并引入 B 改写生成式得 $A \rightarrow cB|f, B \rightarrow d|e$ 。
最终文法为

$$S \rightarrow abA$$

$$A \rightarrow cB|f$$

$$B \rightarrow d|e$$

目录

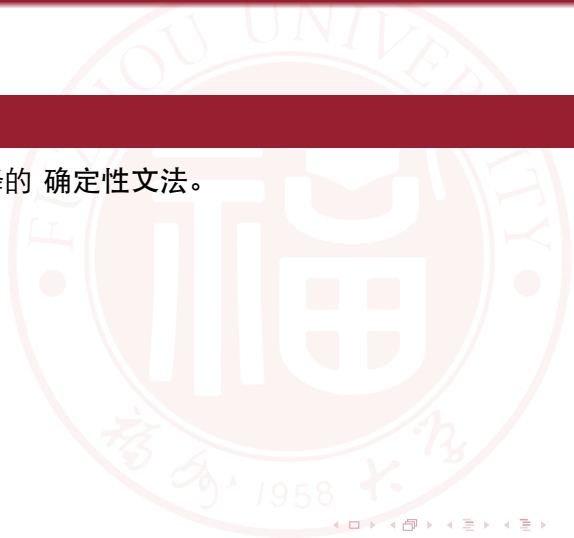
- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
 - 左递归
 - 回溯
 - 综合处理顺序
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业



自顶向下分析：文法预处理流水线

核心目标

- 将原始文法改造为适合递归下降的 确定性文法。



自顶向下分析：文法预处理流水线

核心目标

- 将原始文法改造为适合递归下降的 确定性文法。

具体步骤

① 消除左递归 (Eliminate Left Recursion)

- 目的：解决终止性问题。
- 原因：左递归会导致递归程序进入无限循环，引发栈溢出。
- 例： $A \rightarrow A\alpha \mid \beta$ 必须转换为右递归形式。

自顶向下分析：文法预处理流水线

核心目标

- 将原始文法改造为适合递归下降的 确定性文法。

具体步骤

- ① 消除左递归 (Eliminate Left Recursion)
- ② 提取左公因子 (Left Factoring)
 - 目的：解决确定性问题（消除回溯）。
 - 原因：通过推迟决策，使得分析器仅需查看当前输入符号（Lookahead）即可做出唯一选择。
 - 例： $A \rightarrow \alpha\beta \mid \alpha\gamma$ 转换为 $A \rightarrow \alpha A', A' \rightarrow \beta \mid \gamma$ 。

自顶向下分析：文法预处理流水线

核心目标

- 将原始文法改造为适合递归下降的 确定性文法。

具体步骤

- ① 消除左递归 (Eliminate Left Recursion)
- ② 提取左公因子 (Left Factoring)
- ③ 处理二义性 (Handle Ambiguity)
 - 目的：解决唯一性问题。
 - 原因：二义性文法对同一输入可能产生多棵语法树，需通过引入优先级和结合性规则来改写。
 - 例：经典的“悬空 *else*” 问题或算术表达式的优先级处理。

自顶向下分析：文法预处理流水线

核心目标

- 将原始文法改造为适合递归下降的 确定性文法。

具体步骤

- ① 消除左递归 (Eliminate Left Recursion)
- ② 提取左公因子 (Left Factoring)
- ③ 处理二义性 (Handle Ambiguity)

最终结果

经过上述处理，文法通常能达到 **LL(1)** 条件，从而实现高效的无回溯预测分析。

目录



- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业

递归下降分析的基本思想

- 定义：一种自顶向下的语法分析方法，为文法中的每个非终结符编写一个对应的递归过程（子程序）。
- 核心逻辑：
 - 程序执行的调用序列对应于语法树的自顶向下构造过程。
 - 利用函数的递归调用自动维护分析所需的下推栈。

文法与程序的对应关系

文法成分	程序实现
非终结符 A	函数 <code>void A()</code>
终结符 a	匹配函数 <code>match(a)</code>
产生式右部 XYZ	顺序调用 <code>X(); Y(); Z();</code>
选择 $X \mid Y$	条件分支 <code>if...else...</code> 或 <code>switch</code>
重复 $A \rightarrow \alpha A \mid \beta$	转化为循环 <code>while</code> (针对右递归优化)

基础支持：Match 函数

在递归下降分析中，我们需要一个基础算子来消耗输入序列。

功能描述

- 检查当前读头符号是否与预期一致。
- 若一致，调用词法分析接口获取下一个 token。
- 若不一致，触发错误处理程序。

```
void match(TokenType expected) {  
    if (lookahead == expected) {  
        lookahead = getNextToken();  
    } else {  
        syntax_error(expected);  
    }  
}
```

示例分析：算术表达式

文法改造后：

$$E \rightarrow TE', \quad E' \rightarrow +TE' \mid \epsilon, \quad T \rightarrow FT', \quad T' \rightarrow *FT' \mid \epsilon, \quad F \rightarrow (E) \mid \text{id}$$

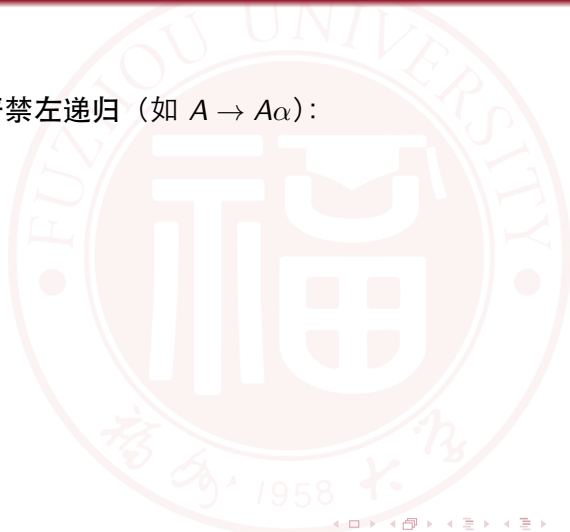
递归子程序实现 (部分)

```
void E() { T(); E_prime(); }
void E_prime() {
    if (lookahead == '+') { match('+'); T(); E_prime(); }
    // epsilon 情况：直接返回（不做任何操作）
}
void F() {
    if (lookahead == '(') { match('('); E(); match(')'); }
    else if (lookahead == ID) { match(ID); }
    else { error(); }
}
```

递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

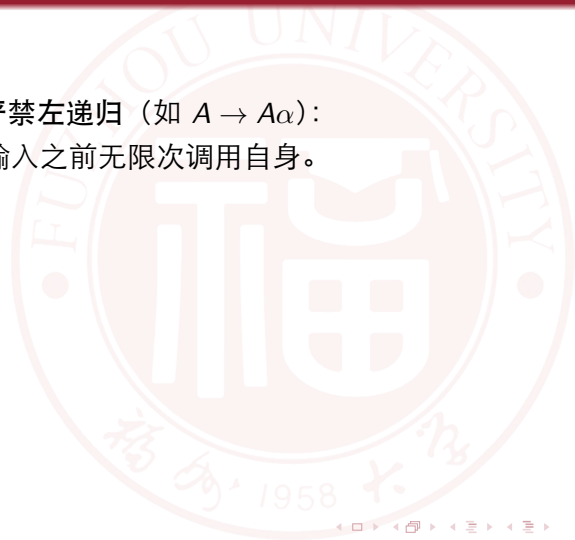


递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

- 原因：函数 $A()$ 会在读取任何输入之前无限次调用自身。

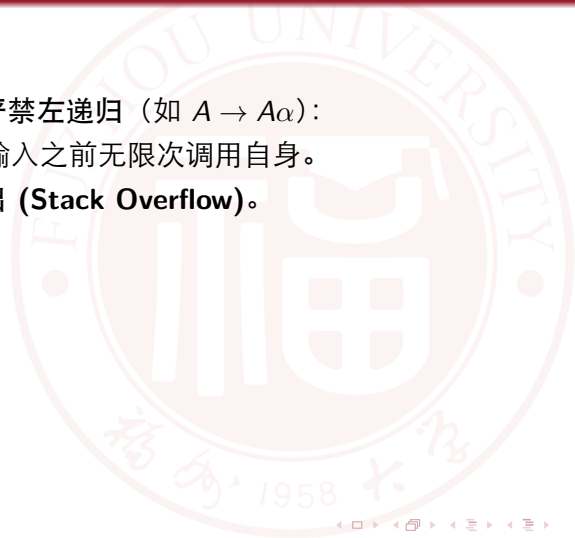


递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

- 原因：函数 $A()$ 会在读取任何输入之前无限次调用自身。
- 后果：直接导致系统程序栈溢出 (**Stack Overflow**)。



递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

- 原因：函数 $A()$ 会在读取任何输入之前无限次调用自身。
- 后果：直接导致系统程序栈溢出 (**Stack Overflow**)。

关于回溯与左公因子的权衡

递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

- 原因：函数 $A()$ 会在读取任何输入之前无限次调用自身。
- 后果：直接导致系统程序栈溢出 (**Stack Overflow**)。

关于回溯与左公因子的权衡

- 允许存在公共左因子：
 - 若存在 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ ，递归下降法可以通过回溯 (Backtracking) 来解决。
 - 缺点：分析器会反复扫描输入串，时间复杂度可能退化为指数级。

递归下降分析的实现前提与局限性

硬性限制：必须消除左递归

不论是否允许回溯，递归下降法都严禁左递归（如 $A \rightarrow A\alpha$ ）：

- 原因：函数 $A()$ 会在读取任何输入之前无限次调用自身。
- 后果：直接导致系统程序栈溢出 (**Stack Overflow**)。

关于回溯与左公因子的权衡

- 允许存在公共左因子：
 - 若存在 $A \rightarrow \alpha\beta_1 \mid \alpha\beta_2$ ，递归下降法可以通过回溯（Backtracking）来解决。
 - 缺点：分析器会反复扫描输入串，时间复杂度可能退化为指数级。
- 消除左公因子（非强制）：
 - 只有为了实现不回溯的预测分析，才强制要求消除左公因子。

广义递归下降的实现逻辑（带回溯）

若要实现带回溯的递归下降，每个子程序需要能够“撤回”操作。

回溯实现示例： $A \rightarrow ab \mid ac$

```
bool A() {  
    int start_pos = current_index; // 记录当前输入指针位置  
    // 尝试第一个候选式 ab  
    if (match('a') && match('b')) return true;  
    // 匹配失败，执行回溯  
    current_index = start_pos;  
    // 尝试第二个候选式 ac  
    if (match('a') && match('c')) return true;  
    return false;  
}
```

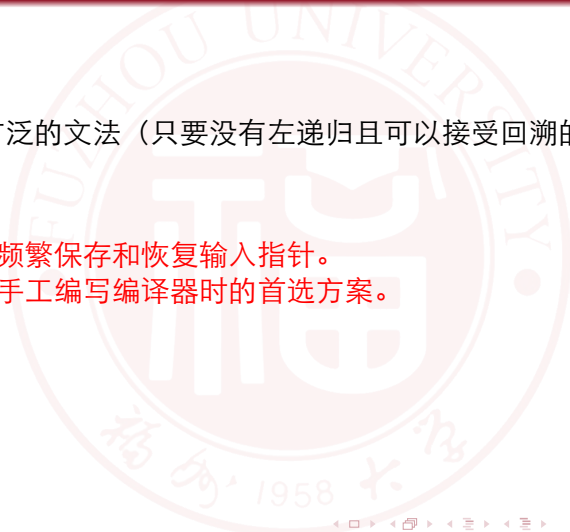
递归下降分析的评价

- 灵活性：能够处理比 LL(1) 更广泛的文法（只要没有左递归且可以接受回溯的代价）。



递归下降分析的评价

- 灵活性：能够处理比 LL(1) 更广泛的文法（只要没有左递归且可以接受回溯的代价）。
- 实现难度：
 - 带回溯 实现较复杂，需要频繁保存和恢复输入指针。
 - 不带回溯 结构清晰，是目前手工编写编译器时的首选方案。



递归下降分析的评价

- 灵活性：能够处理比 LL(1) 更广泛的文法（只要没有左递归且可以接受回溯的代价）。
- 实现难度：
 - 带回溯 实现较复杂，需要频繁保存和恢复输入指针。
 - 不带回溯 结构清晰，是目前手工编写编译器时的首选方案。
- 结论：虽然递归下降法支持回溯，但在实际工程中，我们通常会通过提取左公因子将文法改造为 LL(1)，从而获得高性能的分析器。

目录



- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业

错误处理概述

- 基本目标：
 - ① 准确性：清楚地报告错误出现的位置和性质。
 - ② 恢复能力：从错误中恢复，继续分析剩余部分，尽可能多地发现后续错误。
 - ③ 效率：不应显著降低合法程序的分析速度。
- 副作用（挑战）：
 - 雪崩效应：一个错误引发一系列虚假的伪错误报告。
 - 无限循环：在恢复过程中未能消耗输入符号，导致分析器卡死。

错误的分类

错误类型	典型示例	检测阶段
词法错误	非法字符、字符串未闭合、数值溢出	词法分析
语法错误	括号不匹配、缺少分号、表达式语法错误	语法分析
语义错误	变量未定义、类型不匹配、参数个数不符	语义分析
逻辑错误	无限循环、递归无出口（算法逻辑问题）	运行/测试

语法分析器的职责

语法分析器只能检测出符合文法定义的结构错误，无法判断变量是否有意义。

错误诊察与报告

报告准则

- **精确位置**：指出行号、列号，最好能打印出出错的代码段并用指针标记。
- **清晰描述**：避免晦涩的内部错误码（如 "Error 104"），应使用 "Expected ';' but found 'id'"。

递归下降中的实现

在 match 函数或 switch-default 分支中调用错误处理函数：

- `syntax_error("Missing matching bracket ')'");`

错误处理技术 (1): 恐慌模式 (Panic Mode)

- 核心思想: 发现错误后, 不断跳过输入符号, 直到遇到一个同步令牌 (Synchronizing Token)。
- 同步令牌的选择:
 - 具有清晰边界意义的符号, 如分号 ;、右花括号 }。
 - 当前非终结符的 FOLLOW 集合。

递归下降实现逻辑

- ① 打印错误消息。
- ② 循环调用 getNextToken() 直到当前 lookahead 属于同步集。
- ③ 从该点继续尝试分析。

错误处理技术 (2): 短语级恢复 (Phrase-level Recovery)

- 核心思想：在当前出错的位置进行“局部修复”，使剩下的串合法。
- 常见手段：
 - 插入：发现缺少分号，自动补齐一个分号并继续。
 - 替换：将错误的逗号替换为分号。
 - 删除：删除多余的运算符。

风险

若修复不当，可能导致分析器进入无限死循环（例如不断插入符号却不消耗输入）。
因此，目前主流编译器（如 Clang/GCC）更多地结合使用多种策略。

错误处理技术 (3): 其它方法

- 错误产生式 (Error Productions):
 - 在文法中显式加入常见的错误模式 (如 $E \rightarrow EE$)。
 - 优点: 报错非常精准。
 - 缺点: 文法急剧膨胀, 难以维护。
- 全局校正 (Global Convergence):
 - 寻找代价最小的修改序列将错误串转为正确串。
 - 实际现状: 计算开销过大, 在实际编译器中极少采用。

目录

- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业
 本讲小结
 作业



目录

- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业
 - 本讲小结
 - 作业



本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。



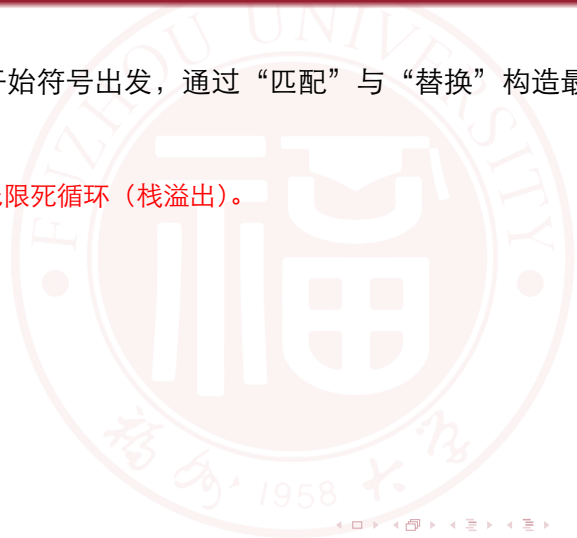
本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：



本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。



本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。

本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。
 - ③ 二义性：导致语法树不唯一。

本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。
 - ③ 二义性：导致语法树不唯一。
- 递归下降分析法：

本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。
 - ③ 二义性：导致语法树不唯一。
- 递归下降分析法：
 - 广义上允许回溯，但工程中倾向于通过改造文法实现确定性分析。

本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。
 - ③ 二义性：导致语法树不唯一。
- 递归下降分析法：
 - 广义上允许回溯，但工程中倾向于通过改造文法实现确定性分析。
 - 文法与程序的映射：非终结符 → 函数；选择 → 分支；重复 → 循环。

本讲小结

- 自顶向下分析的核心：从文法开始符号出发，通过“匹配”与“替换”构造最左推导。
- 文法的“三大杀手”：
 - ① 左递归：导致分析程序陷入无限死循环（栈溢出）。
 - ② 公共左因子：导致分析器频繁回溯，效率退化为指数级。
 - ③ 二义性：导致语法树不唯一。
- 递归下降分析法：
 - 广义上允许回溯，但工程中倾向于通过改造文法实现确定性分析。
 - 文法与程序的映射：非终结符 $\rightarrow$ 函数；选择 $\rightarrow$ 分支；重复 $\rightarrow$ 循环。
- 结论：只有消除左递归和提取左公因子的文法，才能高效地实现无回溯的递归下降分析。

目录



- ① 语法分析概述
- ② 自顶向下分析法及主要矛盾
- ③ 递归下降分析法
- ④ 语法错误处理
- ⑤ 小结与作业
 - 本讲小结
 - 作业

作业练习

① 消除直接左递归

试消除下列文法中的直接左递归：

$$A \rightarrow A\alpha \mid A\beta c \mid \gamma d \mid f$$

② 消除间接左递归

试消除下列文法中的左递归（包含四个非终结符 S, A, B, C ）：

$$S \rightarrow Aa \mid b$$

$$A \rightarrow Bc \mid d$$

$$B \rightarrow Ce \mid f$$

$$C \rightarrow Sg \mid h$$

③ 提取左公因子

对下列文法进行改写，使其能够进行无回溯的自顶向下分析：

$$S \rightarrow iEtS \mid iEtSeS \mid iEtSb \mid a$$

$$E \rightarrow b$$